MLflow Tracking

The MLflow Tracking is an API and UI for logging parameters, code versions, metrics, and output files when running your machine learning code and for later visualizing the results. MLflow Tracking provides Python , REST , R, and Java APIs.

A screenshot of the MLflow Tracking UI, showing a plot of validation loss metrics during model training.

Quickstart

If you haven't used MLflow Tracking before, we strongly recommend going through the following quickstart tutorial.

MLflow Tracking Quickstart

A great place to start to learn the fundamentals of MLflow Tracking! Learn in 5 minutes how to log, register, and load a model for inference.

Ask AI

Concepts

Runs

MLflow Tracking is organized around the concept of **runs**, which are executions of some piece of data science code, for example, a single `python train.py` execution. Each run records metadata (various information about your run such as metrics, parameters, start and end times) and artifacts (output files from the run such as model weights, images, etc).

Models

Models represent the trained machine learning artifacts that are produced during your runs. Logged Models contain their own metadata and artifacts similar to runs.

Experiments

An experiment groups together runs and models for a specific task. You can create an experiment using the CLI, API, or UI. The MLflow API and UI also let you create and search for experiments. See Organizing Runs into Experiments for more details on how to organize your runs into experiments.

Tracking Runs

MLflow Tracking APIs provide a set of functions to track your runs. For example, you can call `mlflow.start_run()` to start a new run, then call Logging Functions such as `mlflow.log_param()` and `mlflow.log_metric()` to log parameters and metrics respectively. Please visit the Tracking API documentation for more details about using these APIs.

python

```python
import mlflow

with mlflow.start_run():
    mlflow.log_param("lr", 0.001)
    # Your ml code
    ...
    mlflow.log_metric("val_loss", val_loss)
```

Alternatively, Auto-logging offers an ultra-quick setup for starting MLflow tracking. This powerful feature allows you to log metrics, parameters, and models without the need for explicit log statements - all you need to do is call `mlflow.autolog()` before your training code. Auto-logging supports popular libraries such as Scikit-learn, XGBoost, PyTorch, Keras, Spark, and more. See Automatic Logging Documentation for supported libraries and how to use auto-logging APIs with each of them.

python

```python
import mlflow

mlflow.autolog()

# Your training code...
```

Ask AI

> **NOTE**
>
> By default, without any particular server/database configuration, MLflow Tracking logs data to the local `mlruns` directory. If you want to log your runs to a different location, such as a remote database and cloud storage, in order to share your results with your team, follow the instructions in the Set up MLflow Tracking Environment section.

Searching Logged Models Programmatically

MLflow 3 introduces powerful model search capabilities through `mlflow.search_logged_models()`. This API allows you to find specific models across your experiments based on performance metrics, parameters, and model attributes using SQL-like syntax.

```python
import mlflow

# Find high-performing models across experiments
top_models = mlflow.search_logged_models(
    experiment_ids=["1", "2"],
    filter_string="metrics.accuracy > 0.95 AND params.model_type = 'RandomForest'",
    order_by=[{"field_name": "metrics.f1_score", "ascending": False}],
    max_results=5,
)

# Get the best model for deployment
best_model = mlflow.search_logged_models(
    experiment_ids=["1"],
    filter_string="metrics.accuracy > 0.9",
    max_results=1,
    order_by=[{"field_name": "metrics.accuracy", "ascending": False}],
    output_format="list",
)[0]

# Load the best model directly
loaded_model = mlflow.pyfunc.load_model(f"models:/{best_model.model_id}")
```

Key Features:

- **SQL-like filtering**: Use `metrics.`, `params.`, and attribute prefixes to build complex queries
- **Dataset-aware search**: Filter metrics based on specific datasets for fair model comparison
- **Flexible ordering**: Sort by multiple criteria to find the best models
- **Direct model loading**: Use the new `models:/<model_id>` URI format for immediate model access

For comprehensive examples and advanced search patterns, see the Search Logged Models Guide.

Ask AI

Querying Runs Programmatically

You can also access all of the functions in the Tracking UI programmatically with MlflowClient.

For example, the following code snippet search for runs that has the best validation loss among all runs in the experiment.

```python
client = mlflow.tracking.MlflowClient()
experiment_id = "0"
best_run = client.search_runs(
    experiment_id, order_by=["metrics.val_loss ASC"], max_results=1
)[0]
print(best_run.info)
# {'run_id': '...', 'metrics': {'val_loss': 0.123}, ...}
```

Tracking Models

MLflow 3 introduces enhanced model tracking capabilities that allow you to log multiple model checkpoints within a single run and track their performance against different datasets. This is particularly useful for deep learning workflows where you want to save and compare model checkpoints at different training stages.

Logging Model Checkpoints

You can log model checkpoints at different steps during training using the `step` parameter in model logging functions. Each logged model gets a unique model ID that you can use to reference it later.

```python
import mlflow
import mlflow.pytorch

with mlflow.start_run() as run:
    for epoch in range(100):
        # Train your model
        train_model(model, epoch)

        # Log model checkpoint every 10 epochs
        if epoch % 10 == 0:
            model_info = mlflow.pytorch.log_model(
                pytorch_model=model,
                name=f"checkpoint-epoch-{epoch}",
                step=epoch,
                input_example=sample_input,
            )

            # Log metrics linked to this specific model checkpoint
            accuracy = evaluate_model(model, validation_data)
            mlflow.log_metric(
                key="accuracy",
                value=accuracy,
                step=epoch,
```

Ask AI

```
            model_id=model_info.model_id,  # Link metric to specific model
            dataset=validation_dataset,
        )
```

Linking Metrics to Models and Datasets

MLflow 3 allows you to link metrics to specific model checkpoints and datasets, providing better traceability of model performance:

```python
# Create a dataset reference
train_dataset = mlflow.data.from_pandas(train_df, name="training_data")

# Log metric with model and dataset links
mlflow.log_metric(
    key="f1_score",
    value=0.95,
    step=epoch,
    model_id=model_info.model_id,  # Links to specific model checkpoint
    dataset=train_dataset,  # Links to specific dataset
)
```

Searching and Ranking Model Checkpoints

Use `mlflow.search_logged_models()` to search and rank model checkpoints based on their performance metrics:

```python
# Search for all models in a run, ordered by accuracy
ranked_models = mlflow.search_logged_models(
    filter_string=f"source_run_id='{run.info.run_id}'",
    order_by=[{"field_name": "metrics.accuracy", "ascending": False}],
    output_format="list",
)

# Get the best performing model
best_model = ranked_models[0]
print(f"Best model: {best_model.name}")
print(f"Accuracy: {best_model.metrics[0].value}")

# Load the best model for inference
loaded_model = mlflow.pyfunc.load_model(f"models:/{best_model.model_id}")
```

Ask AI

Model URIs in MLflow 3

MLflow 3 introduces a new model URI format that uses model IDs instead of run IDs, providing more direct model referencing:

```python
# New MLflow 3 model URI format
model_uri = f"models:/{model_info.model_id}"
loaded_model = mlflow.pyfunc.load_model(model_uri)

# This replaces the older run-based URI format:
# model_uri = f"runs:/{run_id}/model_path"
```

This new approach provides several advantages:

- **Direct model reference**: No need to know the run ID and artifact path
- **Better model lifecycle management**: Each model checkpoint has its own unique identifier
- **Improved model comparison**: Easily compare different checkpoints within the same run
- **Enhanced traceability**: Clear links between models, metrics, and datasets

Tracking Datasets

MLflow offers the ability to track datasets that are associated with model training events. These metadata associated with the Dataset can be stored through the use of the `mlflow.log_input()` API. To learn more, please visit the MLflow data documentation to see the features available in this API.

Explore Runs, Models, and Results

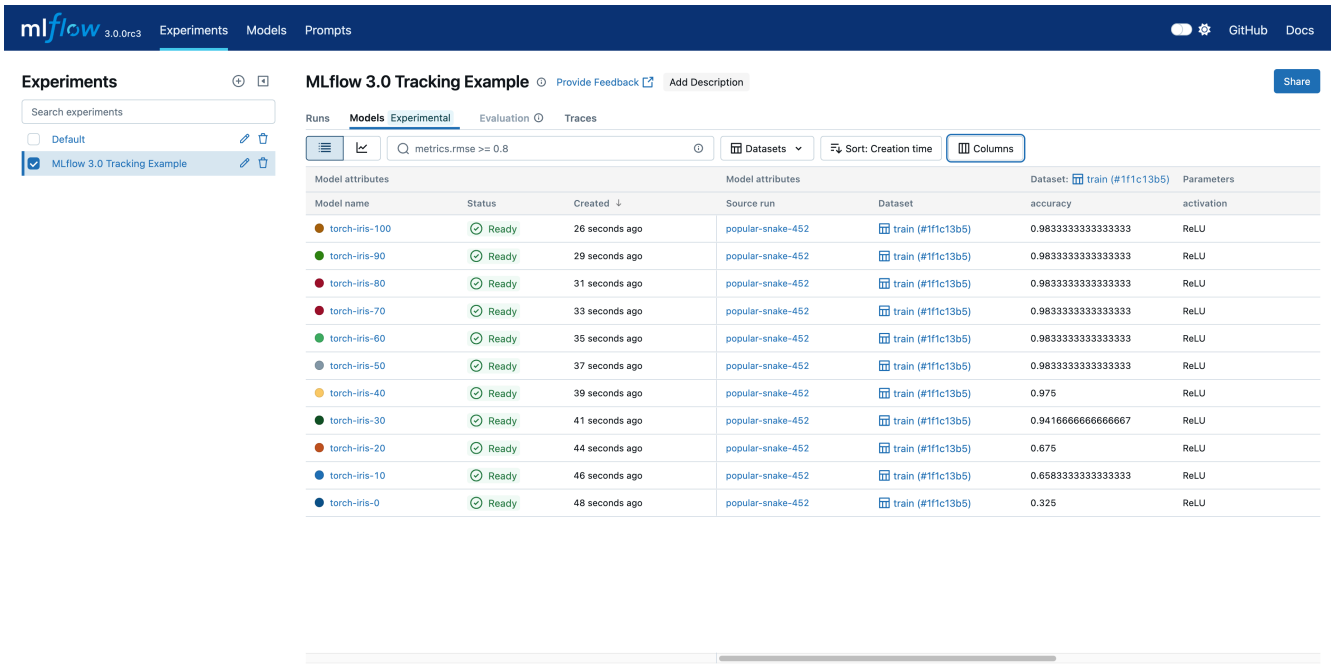
Tracking UI

The Tracking UI lets you visually explore your experiments, runs, and models, as shown on top of this page.

- Experiment-based run listing and comparison (including run comparison across multiple experiments)
- Searching for runs by parameter or metric value
- Visualizing run metrics
- Downloading run results (artifacts and metadata)

These features are available for models as well, as shown below.

Ask AI



A screenshot of the MLflow Tracking UI on the models tab, showing a list of models under the experiment.

If you log runs to a local `mlruns` directory, run the following command in the directory above it, then access http://127.0.0.1:5000 in your browser.

```bash
mlflow server --port 5000
```

Alternatively, the MLflow Tracking Server serves the same UI and enables remote storage of run artifacts. In that case, you can view the UI at `http://<IP address of your MLflow tracking server>:5000` from any machine that can connect to your tracking server.

Set up the MLflow Tracking Environment

> **NOTE**
>
> If you just want to log your experiment data and models to local files, you can skip this section.

MLflow Tracking supports many different scenarios for your development workflow. This section will guide you through how to set up the MLflow Tracking environment for your particular use case. From a bird's-eye view, the MLflow Tracking environment consists of the following components.

Components

MLflow Tracking APIs

You can call MLflow Tracking APIs in your ML code to log runs and communicate [illegible] Tracking Server if necessary.

Ask AI

Backend Store

The backend store persists various metadata for each Run, such as run ID, start and end times, parameters, metrics, etc. MLflow supports two types of storage for the backend: **file-system-based** like local files and **database-based** like PostgreSQL.

Additionally, if you are interfacing with a managed service (such as Databricks or Azure Machine Learning), you will be interfacing with a REST-based backend store that is externally managed and not directly accessible.

Artifact Store

Artifact store persists (typically large) artifacts for each run, such as model weights (e.g. a pickled scikit-learn model), images (e.g. PNGs), model and data files (e.g. Parquet file). MLflow stores artifacts ina a local file (`mlruns`) by default, but also supports different storage options such as Amazon S3 and Azure Blob Storage.

For models which are logged as MLflow artifacts, you can refer the model through a model URI of format: `models:/<model_id>`, where 'model_id' is the unique identifier assigned to the logged model. This replaces the older `runs:/<run_id>/<artifact_path>` format and provides more direct model referencing.

If the model is registered in the MLflow Model Registry, you can also refer to the model through a model URI of format: `models:/<model-name>/<model-version>`, see MLflow Model Registry for details.

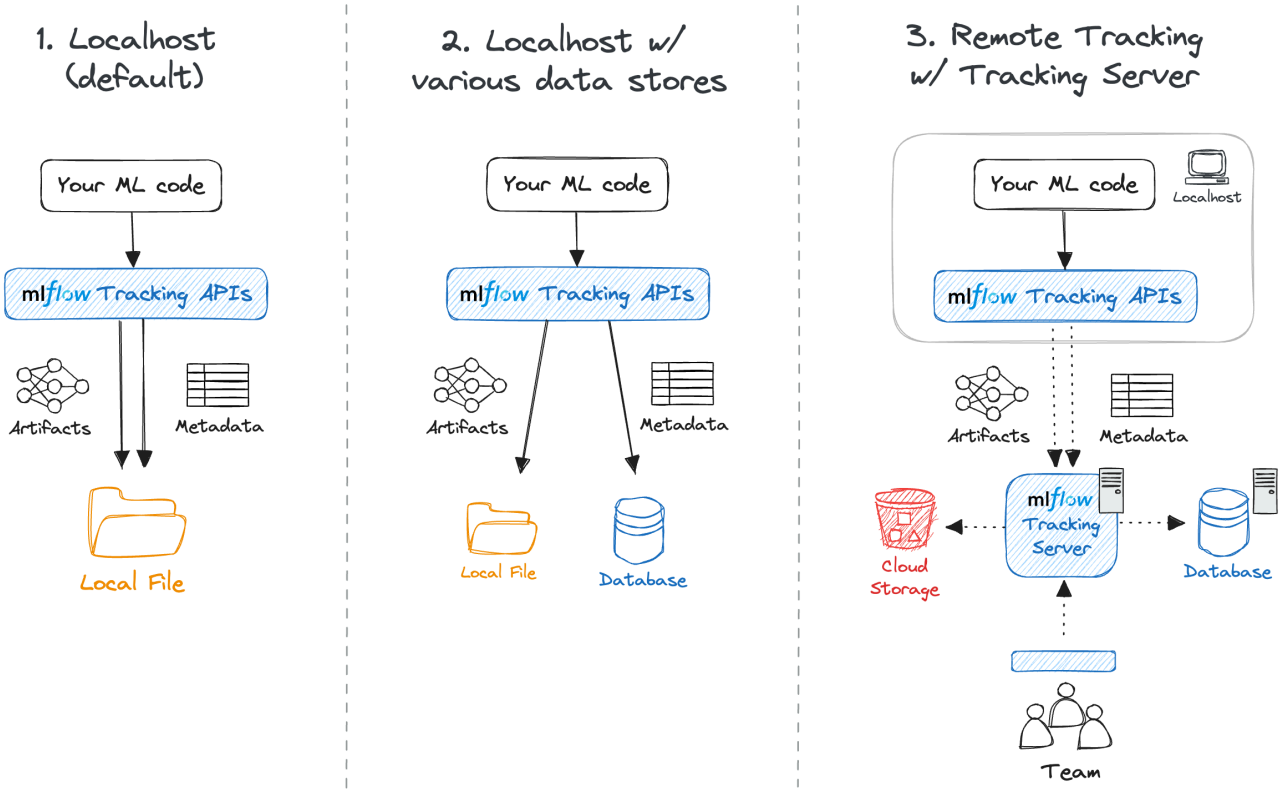
MLflow Tracking Server (Optional)

MLflow Tracking Server is a stand-alone HTTP server that provides REST APIs for accessing backend and/or artifact store. Tracking server also offers flexibility to configure what data to server, govern access control, versioning, and etc. Read MLflow Tracking Server documentation for more details.

Common Setups

By configuring these components properly, you can create an MLflow Tracking environment suitable for your team's development workflow. The following diagram and table show a few common setups for the MLflow Tracking environment.

Ask AI



	1. Localhost (default)	2. Local Tracking with Local Database	3. Remote Tracking with MLflow Tracking Server
Scenario	Solo development	Solo development	Team development
Use Case	By default, MLflow records metadata and artifacts for each run to a local directory, `mlruns`. This is the simplest way to get started with MLflow Tracking, without setting up any external server, database, and storage.	The MLflow client can interface with a SQLAlchemy-compatible database (e.g., SQLite, PostgreSQL, MySQL) for the backend. Saving metadata to a database allows you cleaner management of your experiment data while skipping the effort of setting up a server.	MLflow Tracking Server can be configured with an artifacts HTTP proxy, passing artifact requests through the tracking server to store and retrieve artifacts without having to interact with underlying object store services. This is particularly useful for team development scenarios where you want to store artifacts and experiment metadata in a shared location with proper access control.
Tutorial	QuickStart	Tracking Experiments using a Local Database	Remote Experiment Tracking with MLflow Tracking Server

Other Configuration with MLflow Tracking S

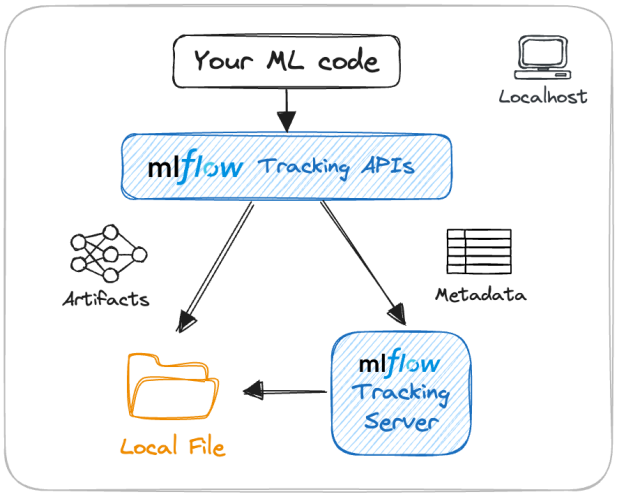
Ask AI

MLflow Tracking Server provides customizability for other special use cases. Please follow Remote Experiment Tracking with MLflow Tracking Server for learning the basic setup and continue to the following materials for advanced configurations to meet your needs.

Local Tracking Server **Artifacts-only Mode** **Direct Access to Artifacts**

Using MLflow Tracking Server Locally

You can of course run MLflow Tracking Server locally. While this doesn't provide much additional benefit over directly using the local files or database, might useful for testing your team development workflow locally or running your machine learning code on a container environment.



FAQ

Can I launch multiple runs in parallel?

Yes, MLflow supports launching multiple runs in parallel e.g. multi processing / threading. See Launching Multiple Runs in One Program for more details.

How can I organize many MLflow Runs neatly?

MLflow provides a few ways to organize your runs:

- Organize runs into experiments - Experiments are logical containers for your runs. You can create an experiment using the CLI, API, or UI.
- Create child runs - You can create child runs under a single parent run to group them together. For example, you can create a child run for each fold in a cross-validation experiment.
- Add tags to runs - You can associate arbitrary tags with each run, which allows you to filter and search runs based on tags.

Can I directly access remote storage without running the Tracking Server?

Yes, while it is best practice to have the MLflow Tracking Server as a proxy for art
development workflows, you may not need that if you are using it for personal p u can
achieve this by following the workaround below:

Ask AI

1. Set up artifacts configuration such as credentials and endpoints, just like you would for the MLflow Tracking Server. See configure artifact storage for more details.
2. Create an experiment with an explicit artifact location,

```python
experiment_name = "your_experiment_name"
mlflow.create_experiment(experiment_name, artifact_location="s3://your-bucket")
mlflow.set_experiment(experiment_name)
```

Your runs under this experiment will log artifacts to the remote storage directly.

How to integrate MLflow Tracking with Model Registry?

To use the Model Registry functionality with MLflow tracking, you **must use database backed store** such as PostgresQL and log a model using the `log_model` methods of the corresponding model flavors. Once a model has been logged, you can add, modify, update, or delete the model in the Model Registry through the UI or the API. See Backend Stores and Common Setups for how to configures backend store properly for your workflow.

How to include additional description texts about the run?

A system tag `mlflow.note.content` can be used to add descriptive note about this run. While the other system tags are set automatically, this tag is **not set by default** and users can override it to include additional information about the run. The content will be displayed on the run's page under the Notes section.

© 2025 MLflow Project, a Series of LF Projects, LLC.

Ask AI